PYTHON INTERVIEW PREPARATION

End-to-end verbal questions • practical programs • OOP • backend and automation

Today's target: Prepare through definitions, explanations, coding exercises, output questions and a practical mini project.

Suggested duration: 6–8 focused hours with breaks. Prioritize fundamentals, collections, functions, OOP, exceptions, file handling and practical coding.

How to use: Answer verbal questions without looking first, type and run programs, then complete the final test without notes.

1. Complete Python interview syllabus

Section	Topics	Priority
1	Fundamentals and data types	Very high
2	Operators, conditions and loops	Very high
3	Strings	Very high
4	Lists, tuples and sets	Very high
5	Dictionaries and comprehensions	Very high
6	Functions, arguments and scope	Very high
7	Lambda, map, filter and reduce	High
8	Iterators and generators	High
9	OOP – four pillars	Very high
10	Exception handling	Very high
11	File handling and context managers	High
12	Modules and packages	High
13	Decorators and closures	High
14	Shallow and deep copy	High
15	Memory management and garbage collection	Medium
16	Threads, processes and async	High
17	Regex and JSON	High
18	Database connectivity and SQL	High
19	REST APIs and FastAPI	Very high
20	Testing, debugging and logging	Very high

2. Python verbal interview questions and answers

A. Python fundamentals

What is Python?
A high-level, general-purpose, dynamically typed language supporting procedural, object-oriented and functional programming.

Main features?
Readable syntax, dynamic typing, standard library, cross-platform support and automatic memory management.

Is Python interpreted or compiled?
Standard CPython compiles source into bytecode, then executes it in the Python virtual machine; calling it simply interpreted is an oversimplification.

What is dynamic typing?
Types belong to runtime objects; a variable name can be reassigned to an object of another type.

Common built-in types?
int, float, complex, bool, str, list, tuple, range, dict, set, frozenset, bytes, bytearray and NoneType.

Mutable vs immutable?
Mutable objects can change after creation (list, dict, set); immutable examples include str, int and tuple.

== vs is?
== checks value equality; is checks object identity.

What is type casting?
Converting a value to another type, such as int('100').

Truthy and falsy?
Falsy examples: False, None, zero, empty string and empty collections. Most other objects are truthy.

What is None?
The singleton value representing absence of a value; its type is NoneType.

B. Operators and control flow

break, continue, pass?
break exits a loop; continue skips to the next iteration; pass is a no-operation placeholder.

for vs while?
for iterates over an iterable; while repeats while a condition is true.

What does range() do?
Produces an immutable sequence of integers, often used in loops.

enumerate()?
Yields index-value pairs from an iterable.

zip()?
Combines iterables position by position; by default stops at the shortest.

and, or, not?
Logical operators for combining or inverting conditions.

C. Strings

Are strings mutable?
No, strings are immutable.

What is slicing?
Extracting a sequence portion with start:stop:step.

split() vs join()?
split() separates a string into parts; join() combines strings from an iterable.

Reverse a string?
text[::-1], or use a loop.

Check palindrome?
Normalize if needed, then compare the text with its reverse.

D. Lists, tuples and sets

List vs tuple?
Lists are mutable; tuples are immutable.

List vs set?
Lists preserve insertion order and allow duplicates; sets contain unique hashable elements and do not guarantee fixed iteration order.

append() vs extend()?
append adds one object; extend adds elements from an iterable.

remove(), pop(), del?
remove(value) removes first matching value; pop(index) removes and returns an item; del removes a binding or indexed/sliced item.

List comprehension?
A concise expression for building a list from an iterable, optionally filtering.

Tuple packing/unpacking?
Packing groups values; unpacking assigns iterable elements to variables.

Set operations?
Union |, intersection &, difference -, symmetric difference ^.

E. Dictionaries

What is a dictionary?
A mutable mapping of hashable keys to values.

Duplicate keys?
No; assigning an existing key updates its value.

Can a list be a key?
No, lists are unhashable; keys must be hashable.

get() vs indexing?
get() can return a default for a missing key; indexing raises KeyError.

setdefault()?
Returns an existing value or inserts the key with a default.

Merge dictionaries?
Use update() or the | merge operator in supported Python versions.

F. Functions

What is a function?
A reusable block of code performing a task.

return vs print()?
return passes a result to the caller; print displays output.

Default arguments?
Parameter values used when the caller omits those arguments.

***args and **kwargs?**
Extra positional arguments become a tuple; extra keyword arguments become a dictionary.

lambda?
A small anonymous function expression.

Recursion?
A function calling itself with a base case to stop.

LEGB?
Local, Enclosing, Global, Built-in lookup order.

Higher-order function?
A function that accepts a function or returns one.

map() vs filter()?
map transforms elements; filter selects elements using a predicate.

G. Iterators and generators

Iterable?
An object from which an iterator can be obtained.

Iterator?
An object that yields values through __next__() and signals exhaustion with StopIteration.

Generator?
An iterator commonly created with yield or a generator expression.

yield vs return?
return ends a function; yield produces a value and suspends execution.

Why generators?
Lazy processing can avoid keeping an entire result set in memory.

H. OOP

What is OOP?
A programming approach organizing code around classes and objects.

Four pillars?
Encapsulation, inheritance, polymorphism and abstraction.

Class?
A blueprint defining attributes and behavior for instances.

Object?
An instance of a class.

self?
Conventional first parameter referring to the current instance.

__init__()?
Initializer called after instance creation; __new__ is responsible for creating the instance.

Class vs instance variable?
A class variable is stored on the class; an instance variable is associated with an object.

Instance/class/static methods?
Instance methods receive self; class methods receive cls; static methods receive neither automatically.

Inheritance?
A class reuses and extends behavior from base classes.

Overriding?
A subclass supplies its own implementation of an inherited method.

Polymorphism?
Different objects respond to a common interface with their own behavior.

Encapsulation?
Bundling data and methods and controlling access through conventions or properties.

Abstraction?
Exposing essential behavior while hiding implementation details.

super()?
Provides access to the next class in method resolution order.

MRO?
The order Python follows when searching classes for attributes and methods.

Composition?
Building a class using other objects; a has-a relationship.

Magic methods?
Special methods such as __init__, __str__, __repr__ and __len__ that integrate objects with Python operations.

I. Exceptions and files

Exception handling?
Handling runtime errors so an application can respond or recover gracefully.

try/except/else/finally?
try contains risky code; except handles matching exceptions; else runs if none occurs; finally runs during cleanup.

raise vs assert?
raise explicitly raises an exception; assert checks a development assumption and may be disabled under optimization.

Why avoid bare except?
It can hide unexpected failures; catch specific exception types.

Context manager?
An object managing setup and cleanup, commonly used with with.

r, w and a modes?
r reads; w writes and truncates; a appends.

read(), readline(), readlines()?
Read all remaining content, one line, or all remaining lines as a list.

J. Advanced Python

Decorator?
A callable wrapping another callable to extend or modify behavior.

Closure?
An inner function retaining access to variables from its enclosing scope.

Shallow vs deep copy?
Shallow copy creates a new outer object but shares nested references; deep copy recursively copies nested objects.

Garbage collection?
Automatic memory management; CPython uses reference counting and cyclic garbage collection.

Module and package?
A module is an importable Python file; a package organizes related modules.

Regex?
A pattern language for matching and manipulating text using re.

JSON?
A text data-interchange format supported by Python's json module.

K. Concurrency and performance

Threads vs processes?
Threads share process memory; processes have separate memory. Processes can use multiple CPU cores for CPU-bound work.

GIL?
In standard GIL-enabled CPython builds it limits simultaneous Python bytecode execution by threads in an interpreter; optional free-threaded builds exist in newer versions.

Asynchronous programming?
Tasks make progress while waiting for I/O, using async and await.

asyncio?
Standard library framework for asynchronous concurrent code.

Improve performance?
Choose suitable algorithms and data structures, profile bottlenecks, avoid repeated work and use concurrency or optimized libraries appropriately.

L. Backend, APIs and automation

REST API?
An architectural style exposing resources through HTTP semantics.

GET vs POST?
GET retrieves a resource; POST submits data, often to create a resource or trigger processing.

FastAPI?
A Python framework for APIs with type-hint-based validation and generated documentation.

Pydantic?
A library for parsing and validating data, often used with FastAPI.

Handle API errors?
Check status codes, catch timeout/connection errors, validate response data, log context and retry only when appropriate.

Logging?
Recording events at DEBUG, INFO, WARNING, ERROR and CRITICAL levels.

Unit testing?
Testing a small isolated unit such as a function or method.

Mocking?
Replacing a dependency with a controlled test double.

Connection pool?
A managed collection of reusable database connections.

Process customers independently?
Use a per-customer error boundary, log failures and continue when business policy permits.

3. Practical coding question bank

Level 1 – Basic programs

1. Reverse a string. Python → nohtyP.

2. Check palindrome.

3. Reverse an integer: 12345 → 54321.

4. Find factorial.

5. Check prime number.

6. Generate Fibonacci series for N terms.

7. Count vowels and consonants.

8. Find largest of three numbers.

9. Sum digits of an integer.

10. Separate even and odd list values.

Level 2 – Collections and strings

11. Find second-largest distinct number.

12. Remove duplicates preserving order.

13. Find duplicate elements.

14. Count element frequency.

15. Find common elements in two lists.

16. Find missing number from 1 to N.

17. Find most frequent character.

18. Check whether strings are anagrams.

19. Group employees by department.

20. Sort dictionary by values.

Level 3 – Functions and advanced Python

21. Factorial function.

22. Accept unlimited positional arguments using *args.

23. Accept keyword arguments using **kwargs.

24. Sort employees by salary using lambda.

25. Use map() to square numbers.

26. Use filter() to select even numbers.

27. Write a generator for even numbers.

28. Decorator to measure execution time.

29. Demonstrate shallow and deep copy.

30. Recursive Fibonacci function.

Level 4 – OOP

31. Employee class with name, ID and salary.

32. BankAccount with deposit and withdrawal.

33. Demonstrate single inheritance.

34. Demonstrate multiple inheritance.

35. Demonstrate overriding.

36. Polymorphism with Email, SMS and Teams notifications.

37. Abstract Payment class.

38. Class method and static method.

39. Encapsulation with private attribute and property.

40. Security Alert Management System.

Level 5 – Files, exceptions and backend

41. Handle division by zero and invalid input.

42. Read a file and count lines.

43. Count ERROR, WARNING and INFO log messages.

44. Read a large file line by line.

45. Validate an API response dictionary.

46. Handle FileNotFoundError.

47. Create and append records to a file.

48. Parse JSON and extract alert details.

49. Validate required API fields.

50. Process multiple customers while isolating individual failures.

4. Practical programs with reference solutions

1. Second-largest distinct number

```
numbers = [10, 20, 50, 30, 70, 70, 20]
unique_numbers = sorted(set(numbers))
if len(unique_numbers) < 2:
    print("Need at least two distinct values")
else:
    print(unique_numbers[-2])
# Output: 50
```

2. Remove duplicates preserving order

```
numbers = [1, 2, 2, 3, 4, 3, 5]
result = []
seen = set()
for num in numbers:
    if num not in seen:
        result.append(num)
        seen.add(num)
print(result)
# [1, 2, 3, 4, 5]
```

3. Word frequency

```
text = "python is easy and python is powerful"
frequency = {}
for word in text.split():
    frequency[word] = frequency.get(word, 0) + 1
print(frequency)
# python: 2, is: 2; other words: 1 each
```

4. Common elements

```
list1 = [1, 2, 3, 4, 5]
list2 = [3, 4, 5, 6, 7]
print(sorted(set(list1) & set(list2)))
# [3, 4, 5]
```

5. Missing number from 1 to N

```
numbers = [1, 2, 3, 5, 6]
n = 6
missing = n * (n + 1) // 2 - sum(numbers)
print(missing)
# 4; assumes distinct values and exactly one missing number.
```

6. Duplicate character frequency

```
text = "programming"
frequency = {}
for char in text:
    frequency[char] = frequency.get(char, 0) + 1
```

```
for char, count in frequency.items():
    if count > 1:
        print(char, count)
# r 2, g 2, m 2
```

7. Sort employees by salary

```
employees = [
    {"name": "John", "salary": 50000},
    {"name": "David", "salary": 70000},
    {"name": "Sam", "salary": 40000}
]
result = sorted(employees,
                key=lambda employee: employee["salary"],
                reverse=True)
for employee in result:
    print(employee)
# David, John, Sam
```

8. Generator for even numbers

```
def even_numbers(n):
    for number in range(0, n + 1):
        if number % 2 == 0:
            yield number
for num in even_numbers(10):
    print(num)
# 0 2 4 6 8 10
```

9. Exception handling

```
try:
    a = int(input("Enter first number: "))
    b = int(input("Enter second number: "))
    result = a / b
except ValueError:
    print("Please enter valid numbers")
except ZeroDivisionError:
    print("Cannot divide by zero")
else:
    print("Result:", result)
finally:
    print("Program completed")
```

10. Count log levels

```
counts = {"INFO": 0, "WARNING": 0, "ERROR": 0}
try:
    with open("application.log", "r") as file:
        for line in file:
            for level in counts:
                if line.startswith(level):
                    counts[level] += 1
                    break
    print(counts)
except FileNotFoundError:
    print("Log file not found")
# Example: {'INFO': 2, 'WARNING': 1, 'ERROR': 2}
```

11. Inheritance, overriding and polymorphism

```
class Alert:
    def __init__(self, alert_id):
        self.alert_id = alert_id
    def send(self):
        print("Sending generic alert")

class EmailAlert(Alert):
    def send(self):
        print("Sending email alert:", self.alert_id)

class SMSAlert(Alert):
    def send(self):
        print("Sending SMS alert:", self.alert_id)

alerts = [EmailAlert("ALT-001"), SMSAlert("ALT-002")]
for alert in alerts:
    alert.send()
```

12. Validate API response

```
def validate_alert(response):
    if not isinstance(response, dict):
        raise ValueError("Response must be a dictionary")
    if response.get("status") != "success":
        raise ValueError("API request was unsuccessful")
    data = response.get("data")
    if not isinstance(data, dict):
        raise ValueError("Invalid API data")
    required_fields = ["alert_id", "severity", "source"]
    for field in required_fields:
        if not data.get(field):
            raise ValueError(f"Missing required field: {field}")
    return data["alert_id"]

response = {
    "status": "success",
    "data": {"alert_id": "ALT-001",
             "severity": "High", "source": "Sentinel"}
}
print(validate_alert(response))
# ALT-001
```

5. Real-world mini project – Security Alert Automation

Requirements: store alerts, validate fields, categorize severity, send notifications polymorphically, update status, handle failures and append results to a log file.

```
class Alert:
    def __init__(self, alert_id, severity, source):
        if severity not in {"Low", "Medium", "High"}:
            raise ValueError("Invalid severity")
        self.alert_id = alert_id
        self.severity = severity
        self.source = source
        self._status = "Open"

    def mark_resolved(self):
        self._status = "Resolved"

    def get_status(self):
        return self._status


class EmailAlert(Alert):
    def send(self):
        print(f"Email sent for {self.alert_id}")


class SMSAlert(Alert):
    def send(self):
        print(f"SMS sent for {self.alert_id}")


def process_alerts(alerts):
    for alert in alerts:
        try:
            if alert.severity == "High":
                alert.send()
            alert.mark_resolved()
            with open("alert_log.txt", "a") as file:
                file.write(f"{alert.alert_id} - {alert.get_status()}\\n")
        except (OSError, ValueError) as error:
            print(f"Failed to process alert: {error}")


alerts = [
    EmailAlert("ALT-001", "High", "Sentinel"),
    SMSAlert("ALT-002", "Medium", "Defender"),
    EmailAlert("ALT-003", "High", "Sentinel")
]
process_alerts(alerts)
for alert in alerts:
    print(alert.alert_id, alert.get_status())
```

Interview discussion: Identify inheritance, overriding, polymorphism, encapsulation, validation, exception handling and context management. Explain adding real API calls, retries, structured logging and per-customer isolation.

6. Final coding test – 60 minutes

Question	Time
Reverse a string without slicing	5 min
Find second-largest distinct number	5 min
Count character frequency	5 min
Find missing number from 1 to N	5 min
Group servers by status	10 min
Write a generator for even numbers	5 min
Employee class with inheritance	10 min
Validate API response with exceptions	10 min
Review and explain complexity	5 min

7. Today's 8-hour preparation schedule

Hour	Focus	Task
1	Fundamentals	Review definitions; write five basic programs.
2	Strings and collections	Practise list, tuple, set, dictionary and string problems.
3	Functions and generators	Practise args, kwargs, lambda, map, filter and yield.
4	OOP	Revise four pillars and complete five OOP exercises.
5	Exceptions and files	Complete log processing, validation and file programs.
6	Advanced Python	Decorators, closures, copying, JSON, modules and concurrency.
7	Backend and automation	Practise API validation, logging, testing and customer processing.
8	Mock interview	Answer 30 verbal questions and solve three coding problems.

8. Final revision checklist

■ Explain data types, mutability and identity.

■ Use strings, lists, tuples, sets and dictionaries confidently.

■ Write functions with positional, keyword, *args and **kwargs arguments.

■ Explain comprehensions, lambda, map, filter and generators.

■ Explain all four OOP pillars with code examples.

■ Use try/except/else/finally and raise specific exceptions.

■ Read and write files safely using with open().

■ Explain decorators, closures, copying and concurrency basics.

■ Parse JSON and validate API response data.

■ Explain logging, testing, mocking and REST API basics.

■ Solve at least 10 coding problems without looking at solutions.

■ Explain approach, time complexity and edge cases.

Final interview tip: For every coding problem, explain your approach, why you selected the data structure, time complexity and edge cases.

This is a focused preparation guide, not a guarantee that every possible interview question is covered. Prioritize understanding and independent coding over memorizing definitions.